Koneru Lakshmaiah Education Foundation

(Deemed to be University)

FRESHMAN ENGINEERING DEPARTMENT

A Project Based Lab ReportOn

Smart Campus Navigation, Facility Monitoring & Resource Analytics System (SCN-FMRA)

SUBMITTED BY:

ID NUMBER	NAME
2500031037	KALANGI NIKITHA
2500031042	CH.VENNELA
2500031010	P.LAVANYA
2500031038	M.LOKESH

UNDER THE GUIDANCE OF

Dr V.PREMALATHA

Assistant Professor, C.S.E

KL UNIVERSITY

Green fields, Vaddeswaram – 522 502

Guntur Dt., AP, India.

DEPARTMENT OF BASIC ENGINEERING SCIENCE-1

CERTIFICATE

This is to certify that the project based laboratory report entitled "**Smart Campus Navigation, Facility Monitoring & Resource Analytics System (SCN-FMRA)**," submitted by **KALANGI NIKITHA**, **CH.VENNELA,P.LAVANYA,M.LOKESH bearing Regd. No. 2500031037, 2500031042, 2500031010, 2500031038** to the **Department of COMPUTER SCIENCE ENGINEERING , KL University** in partial fulfillment of the requirements for the completion of a project based Laboratory in **"PYTHON PROGRAMMING (25SC2007)"** course in I B Tech II Semester, is a Bonafide record of the work carried out by them under my supervision during the academic year 2025 – 2026.

Dr V. PREMALATHA

PROJECT SUPERVISOR

Dr. D. HARITHA

HEAD OF THE DEPARTMENT

ACKNOWLEDGEMENTS

It is great pleasure for me to express my gratitude to our honorable President. **Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the project-based laboratory report.

We express sincere gratitude to our Director **Dr. A. Jagdeesh** for his administration towards our academic growth. We express sincere gratitude to our Principal **Dr. V. Krishna Reddy** for his administration towards our academic growth.

We express sincere gratitude to our HOD-BES-1 **Dr. D. Haritha** for her leadership and constant motivation provided for the successful completion of our academic semester. We record it as my privilege to deeply thank you for providing us with the efficient faculty and facilities to make our ideas into reality.

We express our sincere thanks to our project supervisor **Dr V.PremaLatha** for her novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, We are pleased to acknowledge our indebtedness to all those who devoted themselves directly orindirectly to making this project a success.

Project Associates…

ABSTRACT

Our project entitled “Smart Campus Navigation, Facility Monitoring and Resource Analytics System (SCN-FMRA)”. This project is aimed at developing an intelligent campus management system for students, staff, and administrators. The system helps in navigation, facility usage tracking, maintenance management, and resource analysis within a campus.

The Smart Campus System is a Python-based application that allows users to view available facilities, book rooms, register complaints, and manage campus assets. It provides an efficient way to handle scheduling, avoid conflicts, and maintain proper records of campus resources.

The system can be used within a campus network and can be accessed by authorized users for managing different activities. It also performs data analysis using Pandas and NumPy to understand usage patterns and improve decision-making. Visualization tools such as Matplotlib are used to display data in the form of graphs for better understanding.

The main objective of this project is to provide a smart and efficient solution for campus management by using modern programming techniques and data analysis methods.

INDEX

S.NO	TITLE	PAGE NO
1	Introduction	1
2	Aim of the Project	2
	2.1 Advantages & Disadvantages	2
	2.2 Future Implementation	2
3	Software & Hardware Details	3
4	Flow Chart	4
5	Algorithm for each module	5
6	Implementation	6-16
7	Output/ Screen shots	17-20
8	Conclusion	21

INTRODUCTION

Welcome to the newly designed Smart Campus Navigation, Facility Monitoring & Resource Analytics System (SCN-FMRA), an innovative Python-based platform developed to make campus management smarter, faster, and more efficient. This system is specially designed to help students, faculty, and administrators easily navigate the campus, locate academic blocks and classrooms, and access important facilities without confusion..

SCN-FMRA also provides facility monitoring features to track maintenance activities and resource usage across the campus. By integrating data collection and analytics, it helps administrators make better decisions for improving campus operations. Overall, this project aims to create a more organized, intelligent, and user-friendly campus environment through automation and real-time information.

AIM

- Simplifies campus navigation using an interactive map
- Automates facility monitoring and maintenance tracking
- Reduces manual work and improves efficiency
- Provides data-driven insights for decision-making
- Enhances overall campus management

Advantages

- Easy Navigation – Quickly locate classrooms and facilities.
- Efficient Monitoring – Tracks maintenance and usage digitally.
- Better Decisions – Uses analytics for planning and resource allocation.
- Time Saving – Combines multiple services in one system.

Disadvantages

- Maintenance Requirement – Needs regular updates to keep campus data and maps accurate.
- Dependency on System – If the system fails, users may face difficulty accessing navigation and information.
- Limited Real-Time Accuracy (if offline model) – Without sensors or live integration, updates may not always be real-time.

Future Implementation

- Real-Time Tracking System – Integrate IoT devices or sensors to provide live updates on room occupancy, facility usage, and maintenance alerts.
- AI-Based Smart Suggestions – Implement AI features to suggest optimal routes, predict resource needs, and provide personalized campus assistance for users.

SYSTEM REQUIREMENTS

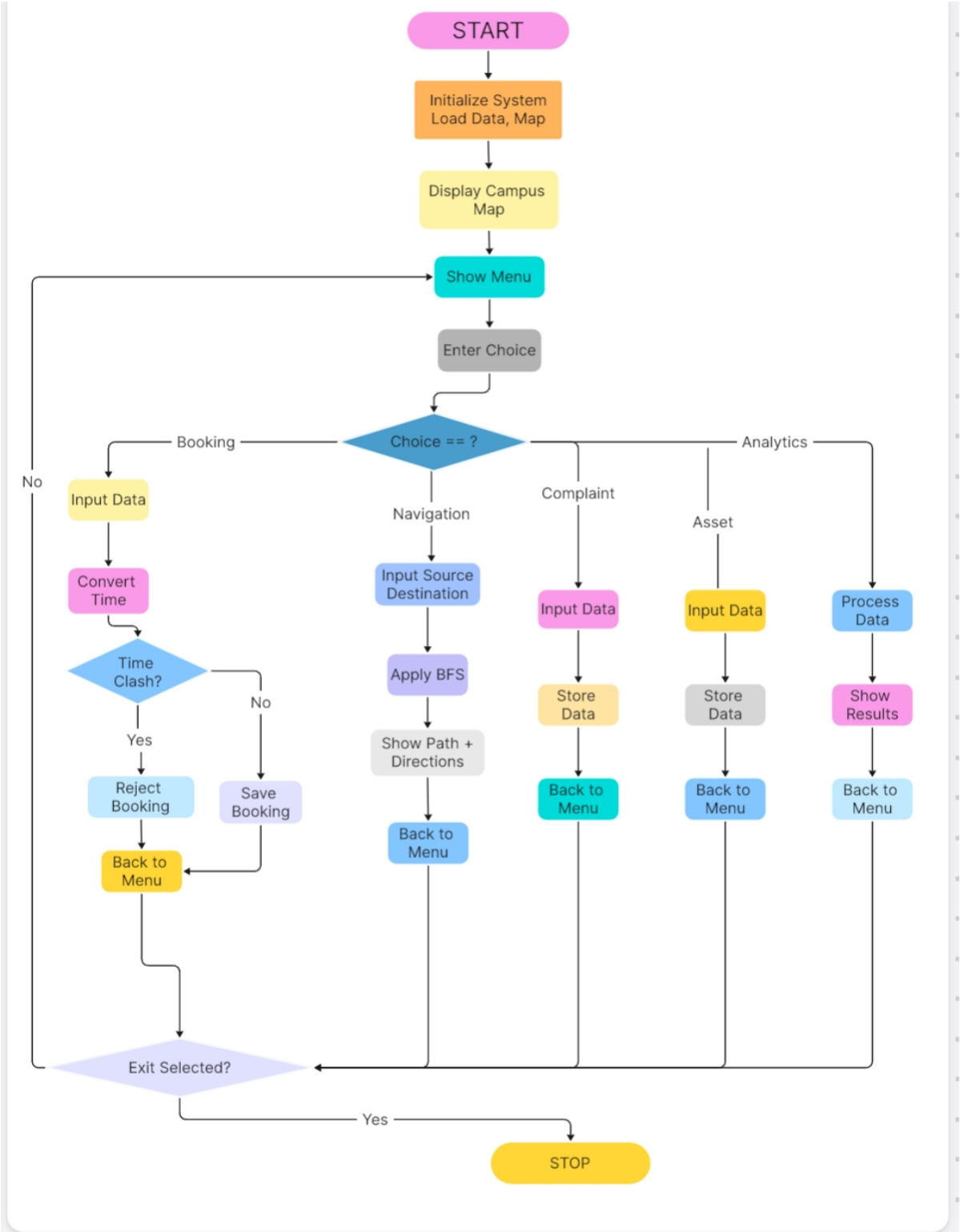
Software Requirements:

- **Programming Language:** Python
- **Libraries Used:** Pandas, NumPy, Matplotlib
- **IDE/Editor:** VS Code / PyCharm / Jupyter Notebook
- **Operating System:** Windows 10 or above / Linux / macOS
- **Visualization Tools:** Matplotlib (for charts and graphs)

Hardware Requirements:

- **Processor:** Intel i3 or above
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Storage:** At least 500 MB free space
- **System Components:** Monitor, Keyboard, Mouse
- **Optional:** Internet connection (for future upgrades like real-time data)

FLOW CHART


START
Initialize System Load Data, Map
Display Campus Map
Show Menu
Enter Choice
Choice == ?
Booking
Navigation
Complaint
Asset
Analytics
No
Input Data
Convert Time
Time Clash?
Yes
No
Reject Booking
Save Booking
Back to Menu
Input Source Destination
Apply BFS
Show Path + Directions
Back to Menu
Input Data
Store Data
Back to Menu
Input Data
Store Data
Back to Menu
Process Data
Show Results
Back to Menu
Exit Selected?
Yes
STOP

ALGORITHM

Campus Navigation

- Input destination
- Find location & shortest path
- Display route

Facility Monitoring

- Input/update facility data
- Store & track status
- Display status

Resource Analytics

- Collect data
- Analyze usage
- Show charts

Maintenance Management

- Report issue
- Track status
- Update after fix

User Interaction

- Take input
- Process request
- Show output

IMPLEMENTATION

The SCN-FMRA project is designed using a modular architecture to ensure better organization and maintainability of the code. The project is divided into different folders such as models, analysis, visualization, storage, and utilities, where each folder handles a specific part of the system functionality. This structured approach improves code readability, reusability, and simplifies future enhancements.

```
scnfmra/
├── __init__.py
├── main.py            ← Entry point
├── models/
│   ├── facility.py     ← Facility + FacilityScheduler
│   ├── booking.py      ← Booking
│   ├── complaint.py    ← Complaint + MaintenanceSystem
│   └── asset.py        ← Asset + InventoryManager
├── analysis/
│   ├── analytics.py
├── visual/
│   └── charts.py
├── utils/
│   ├── navigation.py
├── bookings.csv
├── complaints.csv
├── assets.csv
```

models/facility.py

```
class Facility:
    def __init__(self, facility_id, name, capacity):
        self.facility_id = facility_id
        self.name = name
        self.capacity = capacity

    def display(self):
        return f"{self.facility_id} - {self.name} (Capacity: {self.capacity})"
```

models/booking.py

```
class Booking:
    def __init__(self, booking_id, facility_id, purpose, start, end):
        self.booking_id = booking_id
        self.facility_id = facility_id
        self.purpose = purpose
```

```
        self.start = start
        self.end = end
```

models/complaint.py

```
class Complaint:
    def __init__(self, complaint_id, facility_id, issue, status="Open"):
        self.complaint_id = complaint_id
        self.facility_id = facility_id
        self.issue = issue
        self.status = status
```

models/asset.py (Inheritance)

```
class Asset:
    def __init__(self, asset_id, category, location):
        self.asset_id = asset_id
        self.category = category
        self.location = location


class SmartAsset(Asset):
    def __init__(self, asset_id, category, location, health):
        super().__init__(asset_id, category, location)
        self.health = health
```

utils/navigation.py

```
from collections import deque

def bfs_path(graph, start, goal):
    visited = set()
    queue = deque([(start, [], 0)])

    while queue:
        current, path, dist = queue.popleft()

        if current == goal:
            return path, dist

        if current not in visited:
            visited.add(current)

            for neighbor, direction, d in graph.get(current, []):
                queue.append((
                    neighbor,
                    path + [(current, neighbor, direction, d)],
                    dist + d
                ))
```

```
    return None, 0
```

analysis/analytics.py

```
import pandas as pd
import numpy as np

def facility_usage_analysis(bookings):
    df = pd.DataFrame(bookings)

    if df.empty:
        print("No booking data available")
        return

    print("\n--- Facility Usage ---")
    print(df["facility_id"].value_counts())

    counts = df["facility_id"].value_counts().values
    print("\nAverage Usage:", np.mean(counts))
```

visual/charts.py

```
import pandas as pd
import matplotlib.pyplot as plt

def convert_time(t):
    h, m = map(int, t[:-2].split(":"))
    if "PM" in t and h != 12:
        h += 12
    if "AM" in t and h == 12:
        h = 0
    return h * 60 + m

def show_all_charts(bookings, complaints, assets):
    df_b = pd.DataFrame(bookings)
    df_c = pd.DataFrame(complaints)
    df_a = pd.DataFrame(assets)

    # ---------- 1. BOOKINGS: Most Used Facilities ----------
    if not df_b.empty:
        df_b["facility_id"].value_counts().plot(kind="bar")
        plt.title("Most Used Facilities")
        plt.xlabel("Facility")
        plt.ylabel("Bookings Count")
        plt.show()

    # ---------- LINE CHART: Bookings per Hour ----------
    if not df_b.empty:
        df_b["hour"] = df_b["start"].apply(lambda x: convert_time(x) // 60)
```

```
        hourly_counts = df_b["hour"].value_counts().sort_index()

        plt.plot(hourly_counts.index, hourly_counts.values, marker='o')
        plt.title("Bookings per Hour Trend")
        plt.xlabel("Hour of Day")
        plt.ylabel("Number of Bookings")
        plt.grid()
        plt.show()

    # ---------- 3. ASSETS: Health Status ----------
    if not df_a.empty:
        df_a["health"].value_counts().plot(kind="pie", autopct="%1.1f%%")
        plt.title("Asset Health Status")
        plt.ylabel("")
        plt.show()

    # ---------- 4. COMPLAINTS: Problematic Facilities ----------
    if not df_c.empty:
        df_c["facility_id"].value_counts().plot(kind="bar")
        plt.title("Most Problematic Facilities")
        plt.xlabel("Facility")
        plt.ylabel("Complaints Count")
        plt.show()
```

main.py

```
    from models.facility import Facility
    from models.booking import Booking
    from models.complaint import Complaint
    from models.asset import SmartAsset

    from utils.navigation import bfs_path
    from analysis.analytics import facility_usage_analysis
    from visual.charts import show_all_charts

    import pandas as pd

    # ----------- CSV STORAGE -----------
    def load_csv(filename):
        try:
            df = pd.read_csv(filename)
            return df.to_dict(orient="records")
        except:
            return []

    def save_csv(filename, data):
        df = pd.DataFrame(data)
        df.to_csv(filename, index=False)

    # ----------- DATA -----------
```

```
facilities = [
    Facility("F101", "Classroom A", 60),
    Facility("F102", "Lab B", 30),
    Facility("F103", "Seminar Hall", 100),
    Facility("F104", "Auditorium", 200),
    Facility("F105", "Conference Room", 40),
    Facility("F106", "Computer Lab", 35),
    Facility("F107", "Library Hall", 120)
]
bookings = load_csv("bookings.csv")
complaints = load_csv("complaints.csv")
assets = load_csv("assets.csv")

# ----------- CAMPUS MAP -----------

campus_map = {
    "MainGate": [("Security", "Enter gate", 20)],

    "Security": [
        ("MainGate", "Exit", 20),
        ("AcademicBlock", "Go straight", 50),
        ("Parking", "Turn right", 30)
    ],

    "Parking": [("Security", "Go back", 30)],

    "AcademicBlock": [
        ("Security", "Go back", 50),
        ("Library", "Go straight", 30),
        ("Canteen", "Turn left", 25),
        ("AdminBlock", "Turn right", 20),
        ("Floor1", "Go upstairs", 10),
        ("Floor2", "Go upstairs", 15)
    ],

    "AdminBlock": [
        ("AcademicBlock", "Go back", 20),
        ("Office", "Enter office", 10)
    ],

    "Office": [("AdminBlock", "Exit", 10)],

    "Library": [("AcademicBlock", "Go back", 30)],
    "Canteen": [("AcademicBlock", "Go back", 25)],

    "Floor1": [
        ("AcademicBlock", "Go downstairs", 10),
        ("F101", "Left", 5),
```

```
        ("F102", "Right", 5),
        ("F106", "Computer Lab", 6),
        ("Lab1", "Straight", 8)
    ],

    "Floor2": [
        ("AcademicBlock", "Go downstairs", 15),
        ("F103", "Seminar Hall", 5),
        ("F104", "Auditorium", 6),
        ("F105", "Conference Room", 5)
    ],

    "F101": [("Floor1", "Exit", 5)],
    "F102": [("Floor1", "Exit", 5)],
    "F106": [("Floor1", "Exit", 6)],
    "Lab1": [("Floor1", "Exit", 8)],

    "F103": [("Floor2", "Exit", 5)],
    "F104": [("Floor2", "Exit", 6)],
    "F105": [("Floor2", "Exit", 5)]
}

# ----------- UTIL -----------

def convert_time(t):
    time_part = t[:-2]
    period = t[-2:]

    hours, minutes = map(int, time_part.split(":"))

    if period == "PM" and hours != 12:
        hours += 12
    if period == "AM" and hours == 12:
        hours = 0

    return hours * 60 + minutes

# ----------- FUNCTIONS -----------

def show_facilities():
    print("\n--- Facilities ---")
    for f in facilities:
        print(f.display())

def add_booking():
    try:
        bid = input("Booking ID: ")
        fid = input("Facility ID: ")
```

```
        purpose = input("Purpose: ")

        start = input("Start Time (7:10AM): ")
        end = input("End Time (8:50AM): ")

        start_time = convert_time(start)
        end_time = convert_time(end)

        if start_time >= end_time:
            print("Invalid time range")
            return

        # TIME CLASH CHECK
        for b in bookings:
            if b["facility_id"] == fid:
                existing_start = convert_time(b["start"])
                existing_end = convert_time(b["end"])

                if not (end_time <= existing_start or start_time >= existing_end):
                    print("Time clash! Already booked.")
                    return

        b = Booking(bid, fid, purpose, start, end)
        bookings.append(vars(b))
        save_csv("bookings.csv", bookings)
        print("Booking Saved to CSV")

    except Exception as e:
        print("Error:", e)

def add_complaint():
    cid = input("Complaint ID: ")
    fid = input("Facility ID: ")
    issue = input("Issue: ")

    c = Complaint(cid, fid, issue)
    complaints.append(vars(c))

    save_csv("complaints.csv", complaints)

    print("Complaint Saved to CSV")

def add_asset():
    aid = input("Asset ID: ")
    category = input("Category: ")
    location = input("Location: ")
    health = input("Health Status: ")
```

```
        a = SmartAsset(aid, category, location, health)

        assets.append(vars(a))

    save_csv("assets.csv", assets)

    print("Asset Saved to CSV")

def analytics():
    facility_usage_analysis(bookings)

def charts():
    show_all_charts(bookings, complaints, assets)

def navigate():
    print("\n====== CAMPUS NAVIGATION SYSTEM ======")

    # Show available locations clearly
    print("\nAvailable Locations:")
    for loc in campus_map.keys():
        print("•", loc)

    start = input("\nEnter Start Location: ").strip()
    end = input("Enter Destination: ").strip()

    # Validation
    if start not in campus_map or end not in campus_map:
        print("\n Invalid location entered. Please try again.")
        return

    if start == end:
        print("\n You are already at the destination.")
        return

    # Find path
    path, total_dist = bfs_path(campus_map, start, end)

    if path:
        print("\nNavigation Path:\n")

        step_no = 1
        for src, dest, direction, dist in path:
            print(f"Step {step_no}: {src} → {dest}")
            print(f"   Direction : {direction}")
            print(f"   Distance  : {dist} meters\n")
            step_no += 1

        print(" Destination Reached!")
        print(f" Total Distance: {total_dist} meters")
```

```
    else:
        print("\n No path found between locations.")


import networkx as nx
import matplotlib.pyplot as plt

def show_visual_map():
    G = nx.Graph()

    # Add edges
    for node in campus_map:
        for neighbor, direction, dist in campus_map[node]:
            G.add_edge(node, neighbor, weight=dist)
    pos = {
        "MainGate": (0, 0),
        "Security": (0, 1),
        "Parking": (2, 1),

        "AcademicBlock": (0, 3),
        "AdminBlock": (2, 3),
        "Office": (3, 3),

        "Library": (-2, 4),
        "Canteen": (-2, 2),

        "Floor1": (-1, 5),
        "Floor2": (1, 5),

        "F101": (-2, 6),
        "F102": (-1, 6),
        "F106": (0, 6),
        "Lab1": (-1, 7),

        "F103": (1, 6),
        "F104": (2, 6),
        "F105": (3, 6)
    }

    plt.figure(figsize=(14, 10))
    node_colors = []
    for node in G.nodes():
        if node.startswith("F"):
            node_colors.append("lightgreen")   # rooms
        elif "Floor" in node:
            node_colors.append("pink")         # floors
        elif node in ["Library", "Canteen"]:
            node_colors.append("orange")       # facilities
        elif node in ["AdminBlock", "Office"]:
```

```
            node_colors.append("violet")       # admin
        else:
            node_colors.append("skyblue")      # general

    # Draw graph
    nx.draw_networkx_nodes(G, pos, node_size=2200, node_color=node_colors)
    nx.draw_networkx_edges(G, pos, width=2)
    nx.draw_networkx_labels(G, pos, font_size=10, font_weight='bold')

    # Distance labels
    edge_labels = nx.get_edge_attributes(G, 'weight')
    nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=8)

    plt.title("SMART CAMPUS MAP (REALISTIC VIEW)", fontsize=16)
    plt.axis('off')
    plt.tight_layout()
    plt.show()

def main():
    while True:
        print("\n====== SMART CAMPUS SYSTEM ======")
        print("1. Show Facilities")
        print("2. Add Booking")
        print("3. Add Complaint")
        print("4. Add Asset")
        print("5. Analytics")
        print("6. Charts")
        print("7. Navigation")
        print("8. Visual Campus Map")
        print("9. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            show_facilities()
        elif choice == "2":
            add_booking()

        elif choice == "3":
            add_complaint()
        elif choice == "4":
            add_asset()
        elif choice == "5":
            analytics()
        elif choice == "6":
            charts()
        elif choice == "7":
            navigate()
        elif choice == "8":
```

```
            show_visual_map()
        elif choice == "9":
            print("Exiting...")
            break

if __name__ == "__main__":
    main()
```

OUTPUT SCREEN SHOTS

Screen Shots:

```
C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe C:\Users\HP\Desktop\SCNFMRA\main.py

====== SMART CAMPUS SYSTEM ======
1. Show Facilities
2. Add Booking
3. Add Complaint
4. Add Asset
5. Analytics
6. Charts
7. Navigation
8. Visual Campus Map
9. Exit
Enter choice: 1

--- Facilities ---
F101 - Classroom A (Capacity: 60)
F102 - Lab B (Capacity: 30)
F103 - Seminar Hall (Capacity: 100)
F104 - Auditorium (Capacity: 200)
F105 - Conference Room (Capacity: 40)
F106 - Computer Lab (Capacity: 35)
F107 - Library Hall (Capacity: 120)
```

```
====== SMART CAMPUS SYSTEM ======
1. Show Facilities
2. Add Booking
3. Add Complaint
4. Add Asset
5. Analytics
6. Charts
7. Navigation
8. Visual Campus Map
9. Exit
Enter choice: 2
Booking ID: B11
Facility ID: F107
Purpose: Library
Start Time (7:10AM): 5:00AM
End Time (8:50AM): 7:00AM
Booking Saved to CSV
```

```
4. Add Asset
5. Analytics
6. Charts
7. Navigation
8. Visual Campus Map
9. Exit
Enter choice: 3
Complaint ID: A1
Facility ID: F103
Issue: Fan not working
Complaint Saved to CSV

====== SMART CAMPUS SYSTEM ======
1. Show Facilities
2. Add Booking
3. Add Complaint
4. Add Asset
5. Analytics
6. Charts
7. Navigation
8. Visual Campus Map
9. Exit
Enter choice: 4
Asset ID: A6
Category: Speaker
Location: F102
Health Status: Needs repair
Asset Saved to CSV
```

```
====== SMART CAMPUS SYSTEM ======
1. Show Facilities
2. Add Booking
3. Add Complaint
4. Add Asset
5. Analytics
6. Charts
7. Navigation
8. Visual Campus Map
9. Exit
Enter choice: 5

--- Facility Usage ---
facility_id
F101    2
F103    2
F104    2
F105    2
F102    1
F106    1
F107    1
Name: count, dtype: int64
```

```
9. Exit
Enter choice: 7

====== CAMPUS NAVIGATION SYSTEM ======

Available Locations:
• MainGate
• Security
• Parking
• AcademicBlock
• AdminBlock
• Office
• Library
• Canteen
• Floor1
• Floor2
• F101
• F102
• F106
• Lab1
• F103
• F104
• F105
```

```
• F104
• F105

Enter Start Location: Library
Enter Destination: Lab1

📍 Navigation Path:

Step 1: Library → AcademicBlock
   Direction : Go back
   Distance  : 30 meters

Step 2: AcademicBlock → Floor1
   Direction : Go upstairs
   Distance  : 10 meters

Step 3: Floor1 → Lab1
   Direction : Straight
   Distance  : 8 meters

✅ Destination Reached!
📏 Total Distance: 48 meters
```

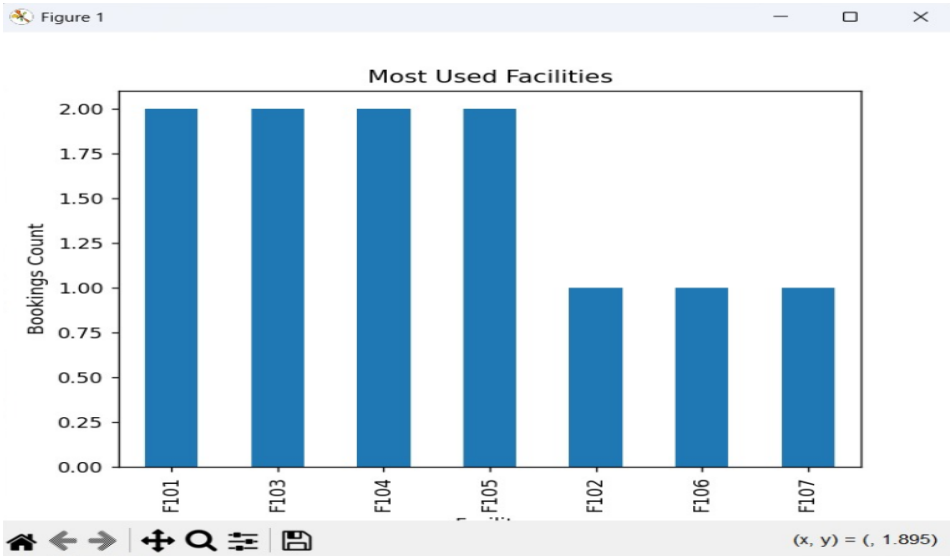
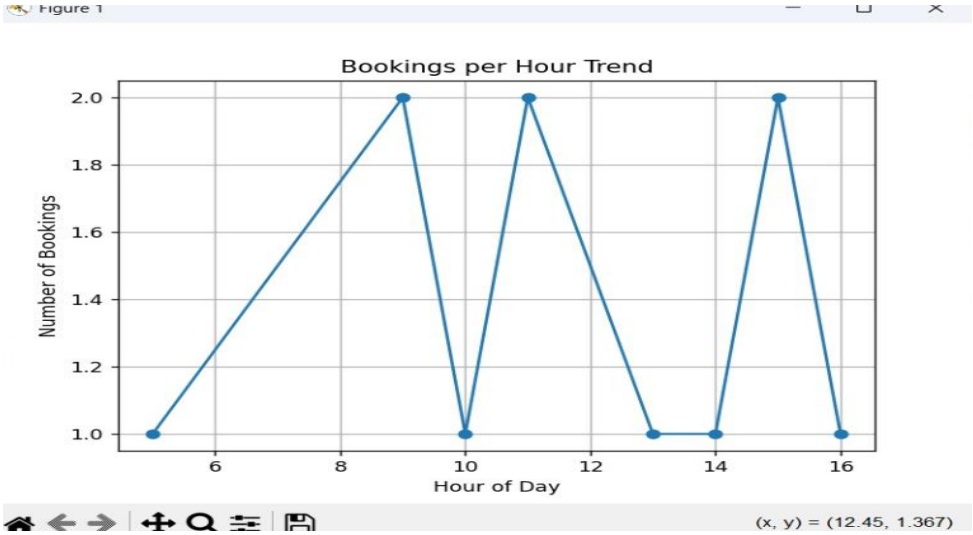


Figure 1
Most Used Facilities
Bookings Count
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
F101
F103
F104
F105
F102
F106
F107
(x, y) = (, 1.895)


Bookings per Hour Trend
Number of Bookings
2.0
1.8
1.6
1.4
1.2
1.0
6
8
10
12
14
16
Hour of Day
(x, y) = (12.45, 1.367)

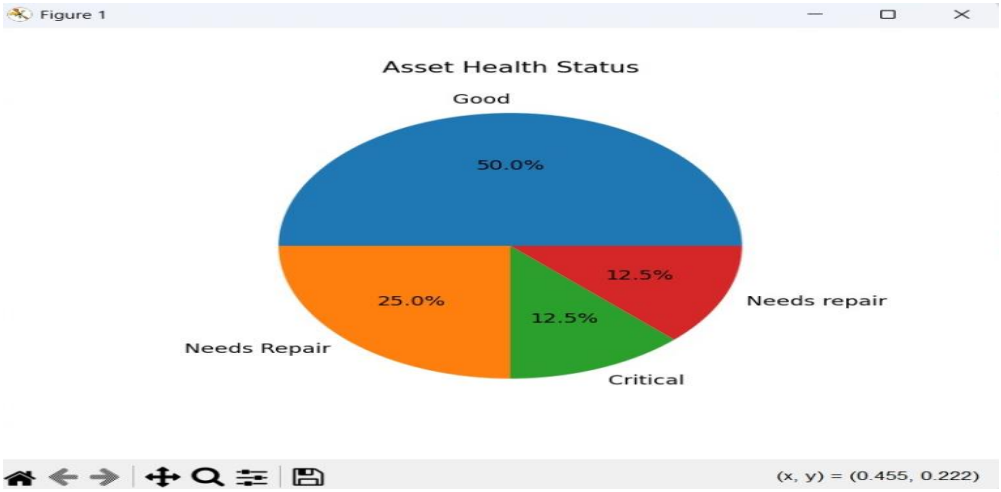


Figure 1
Asset Health Status
Good
50.0%
12.5%
Needs repair
25.0%
12.5%
Needs Repair
Critical
(x, y) = (0.455, 0.222)

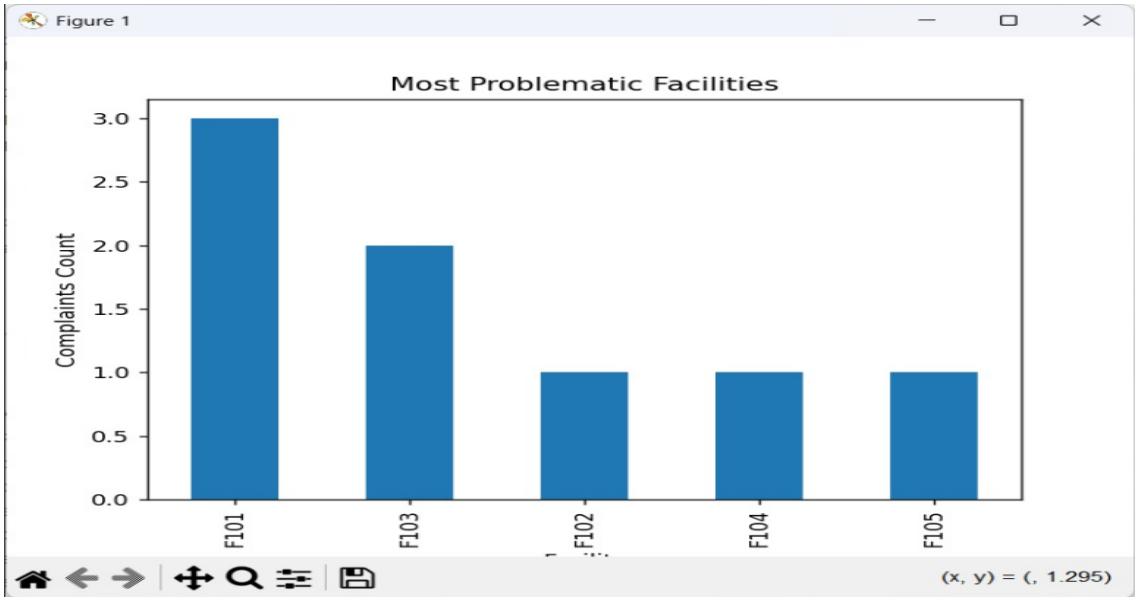


Figure 1
Most Problematic Facilities
Complaints Count
3.0
2.5
2.0
1.5
1.0
0.5
0.0
F101
F103
F102
F104
F105
(x, y) = (, 1.295)

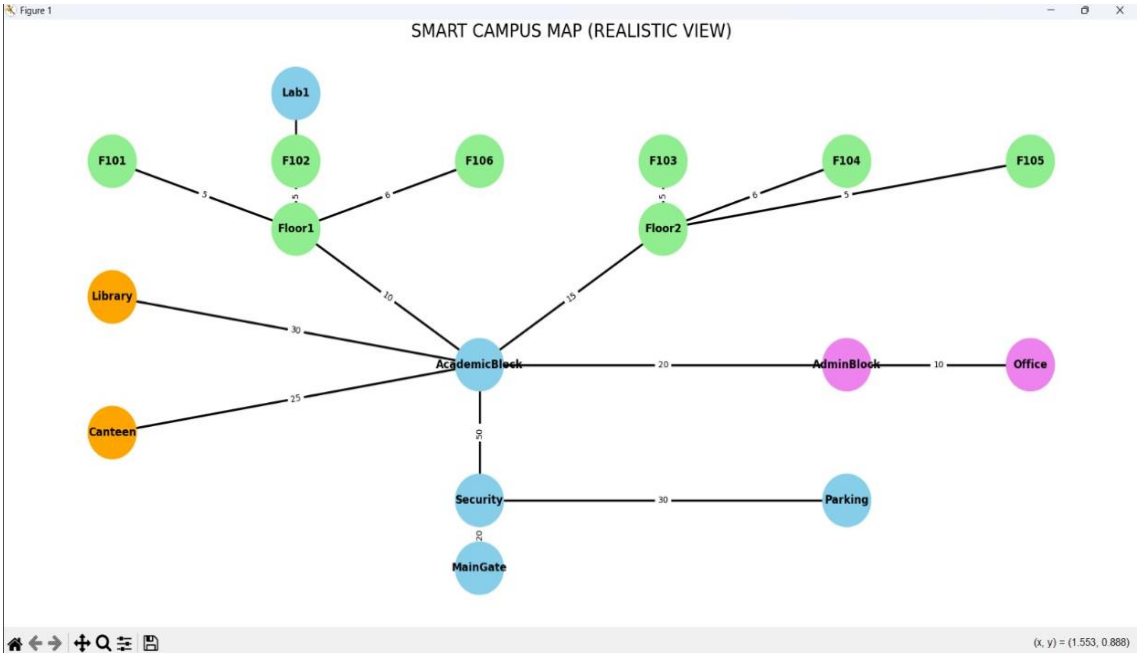


Figure 1
SMART CAMPUS MAP (REALISTIC VIEW)
Lab1
F101
F102
F106
F103
F104
F105
Floor1
Floor2
Library
AcademicBlock
AdminBlock
Office
Canteen
Security
Parking
MainGate
(x, y) = (1.553, 0.888)

CONCLUSION

The Smart Campus Navigation, Facility Monitoring & Resource Analytics System (SCN-FMRA) successfully provides an integrated solution for improving campus management through Python-based automation. The system simplifies campus navigation by helping users easily locate academic blocks and classrooms using an interactive map.

It also enhances facility monitoring by tracking usage and maintenance activities, reducing manual effort and improving efficiency. Additionally, the resource analytics module helps in analyzing data and generating useful insights for better decision-making by administrators.

Overall, SCN-FMRA improves organization, saves time, and contributes to building a smarter and more efficient campus environment.